CMSC 420: Disjoint Set Data Structures

Justin Wyss-Gallifent

January 22, 2026

1 Problems **2**

1 Problems

1. Draw a disjoint forest corresponding to the following disjoint subsets. There are many answers.

$$\{1, 5, 8, 10, 3\}, \{4, 6, 9\}, \{2, 7\}$$

Solution:
One example is:

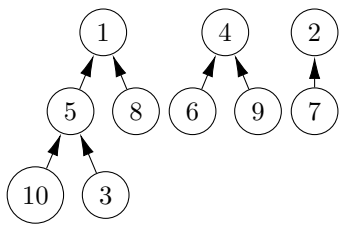

Figure 1: Disjoint Set Data Structure

2. Suppose a graph G starts with n nodes and no edges. We represent it by a disjoint set forest. Assume we are using path compression and weighted union.

   (a) Assume $n = 2^k$ for some positive integer k. Prove by induction on k that it is possible to find a sequence of edges which could be added to the graph and which would result in the disjoint set forest finally ending up as a single tree of height exactly k.

   Solution:
   For $k = 1$ if we start with $2^k = 2$ nodes then simply taking the union will result in a single tree of height 1.
   Assume the statement is true for some k. Suppose we have 2^{k+1} nodes. We split them into two sets of 2^k each which by the induction hypothesis we can find a sequence of edges which results in two trees of height k each. We then union those trees by adding an edge which joins the graph nodes represented by the disjoint set forest root nodes. This avoids path compression and results in a tree of height $k + 1$.
   Note: It's critical to point out that we add an edge which joins the graph nodes representing the disjoint set forest root nodes. If we use some other graph nodes then path compression may mess up the height of one or both trees.

   (b) Is it possible to find a sequence of edges which could be added to the graph and which would result in the disjoint set forest ending up as a single tree of height exactly $n - 1$? Why or why not? Your answer does not need to be particularly formal, a general outline will suffice.

   Solution:
   No. In order to make it happen we'd need every union to attach one tree at the very bottom of the other. However weighted unions don't work this way.

3. Suppose Kruskal's Algorithm (see the notes) is run on the following graph:

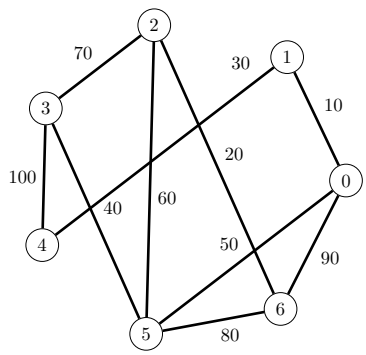

Figure 2: Graph for the Problem

Draw the disjoint forest at the start and then after each edge has been added. Use path compression and weighted union. Your pictures should be after these have occurred.

Solution:
Start:

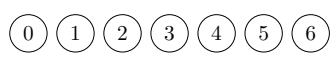

Figure 3: Start

Add edge $(0, 1)$:

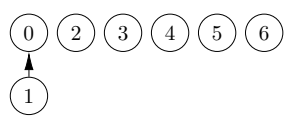

Figure 4: After union(0,1)

Add edge $(2, 6)$:

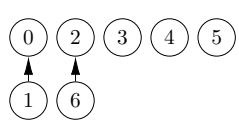

Figure 5: After union(2,6)

Add edge $(1, 4)$. This is a weighted union:

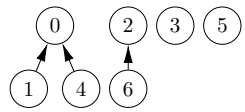

Figure 6: After union(1,4)

Add edge $(3, 5)$:

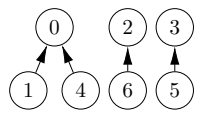

Figure 7: After union(3,5)

Add edge $(0, 5)$. This is a weighted union:

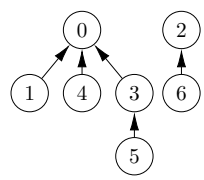

Figure 8: After union(0,5)

Add edge $(5, 2)$. Observe that getting the representative of 5 invokes meaningful path compression. Also this is a weighted union:

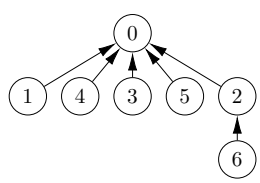

Figure 9: After union(5,2)

Note: There are options here, for example adding $(0, 1)$ at the start could have resulted in 1 above 0.

4. Suppose Kruskal's Algorithm (see the notes) is run on the following graph:

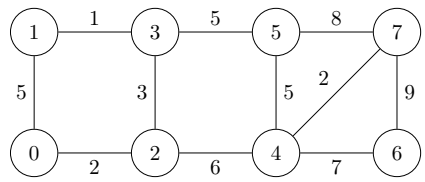

Figure 10: Graph for the Problem

Draw the disjoint forest at the start and then after each edge has been added. Use path compression and weighted union when appropriate. Your pictures should be after these have occurred.

Note: For consistency treat all edges as $x - y$ with $x < y$ and when choices are available choose the edge with smallest x. Moreover `union(x,y)` should always be called with $x < y$.

Solution:
Start:

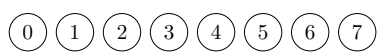

Figure 11: Start

Call `union(1,3)`:

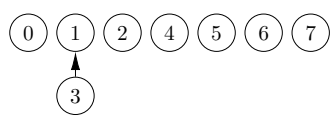

Figure 12: After union(1,3)

Call `union(0,2)`:

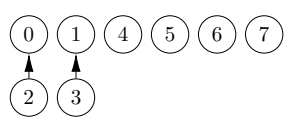

Figure 13: After union(0,2)

Call `union(4,7)`:

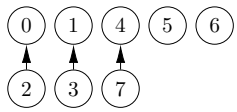

Figure 14: After union(4,7)

Call `union(2,3)`:

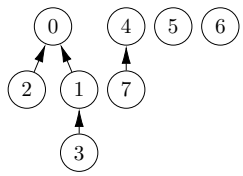

Figure 15: After union(2,3)

Call `union(3,5)`:

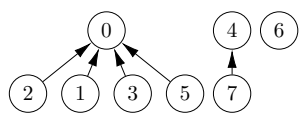

Figure 16: After union(3,5)

Call `union(4,5)`:

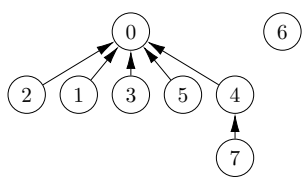

Figure 17: After union(4,5)

Call `union(4,6)`:

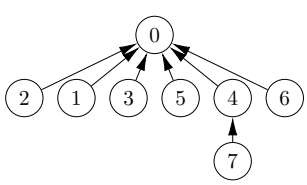

Figure 18: After union(4,6)

5. A classic way to store a disjoint set forest where the keys are nonnegative integers $0, ..., n-1$ is simply via a list. We create a list A of length n such that for each key $0 \leq k \leq n-1$ the entry $A[k]$ equals the parent key.

   (a) Draw the disjoint set forest represented by the list:

   $$A = [6, 1, 3, 3, 2, 6, 6, 5, 8, 8, 5, 6, 3]$$

   Solution:
   It would be:

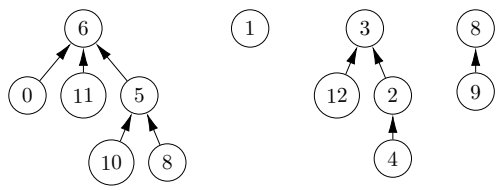

Figure 19: Disjoint Set Forest Solution

   (b) With this storage mechanism write pseudocode for the path-compression version of `findrep` coded as `findrep(A,x)`.

   Solution:
   We have something like:

```
function findrep(A,x):
if A[x] == x:
    return(x)
else:
    return(A[x] = findrep(A,A[x])
end
end
```

6. Given the following disjoint set forest:

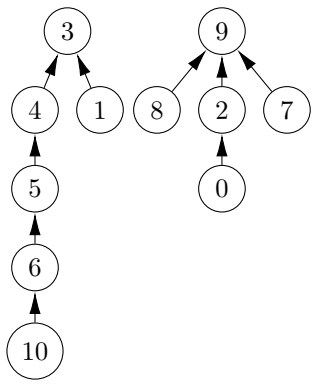

Figure 20: A Disjoint Set Forest

Questions are non-cumulative and assume path compression and weighted union.

(a) Suppose we call `findrep(x)` for x in the left tree and the resulting height of the left tree is unchanged. What could x be?

What could x be?	$1, 3, 4$

(b) Draw the resulting structure when we perform `union(6,0)`.
Solution:

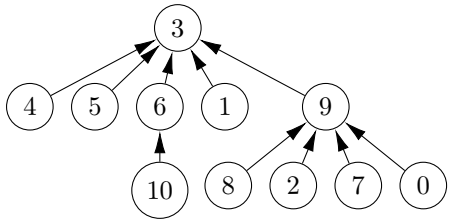

Figure 21: After union(6,0)

(c) Suppose we randomly choose an x in the left tree and we randomly choose y in the right tree (all choices equally likely) and we perform `union(x,y)` what is the probability that the resulting tree has each of the following heights. Some have been answered.

Height ≤ 1	0
Height 2	2/30
Height 3	13/30
Height 4	15/30
Height ≥ 5	0

7. Suppose a 0-indexed list `A` represents a disjoint set forest. Consider the following algorithm:

```
for i = 0 to len(A)-1 inclusive:
    while A[A[i]] != A[i]:
        A[i] = A[A[i]]
    end while
end for
```

What would this algorithm do to the disjoint set forest? Your answer should require at most a sentence or two.

Solution:
It would compress each tree so that every node points to the tree's original root.

8. Consider the following weighted graph:

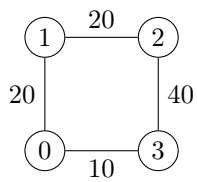

Figure 22: Small Weighted Graph

We run Kruskal's algorithm which iterates three times. What would the corresponding disjoint set forest look like after each iteration?

Note: There are multiple answers, any valid answer will be accepted.

Solution:
There are two structure patterns that can result:

(a) METHOD 1:
After First: 0 and 3 are joined.
After Second: 1 and 2 are joined.
After Third: One of the two pairs is placed under the root of the other.
Note: The result of this has height 2.

(b) METHOD 2:
After First: 0 and 3 are joined.
After Second: 1 is placed under the root of the $0 - 3$ tree.
After Third: 2 is placed under the root of the other tree.
Note: The result of this has height 1.

9. Suppose a graph G starts with $n \geq 3$ nodes and no edges. We represent it by a disjoint set forest. Assume we are using path compression and weighted union. Is it possible to find a sequence of edges which could be added to the graph and which would result in the disjoint set forest ending up as a single tree of height exactly $n-1$? Why or why not? Your answer does not need to be particularly formal, a general outline will suffice.

   Solution:
   No. In order to make it happen we'd need every union to attach one tree at the very bottom of the other. However weighted unions don't work this way.

10. Given the following disjoint set forest:

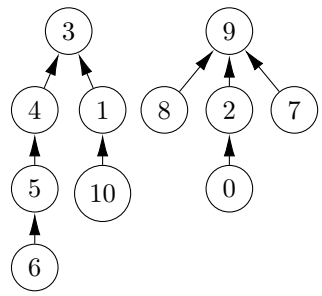

Figure 23: Disjoint Set Forest

Questions are non-cumulative. Assume path compression and weighted union.

(a) Suppose we call `findrep(x)` for some x and the height of at least one tree changes. What could x be?

What could x be?	$0, 5, 6$

(b) Suppose we randomly choose an x in the left tree and we randomly choose y in the right tree (all choices equally likely) and we perform `union(x,y)`. What is the probability that the resulting tree has each of the following heights. Some have been answered.

Height ≤ 1	0
Height 2	$2/30$
Height 3	$28/30$
Height ≥ 4	0